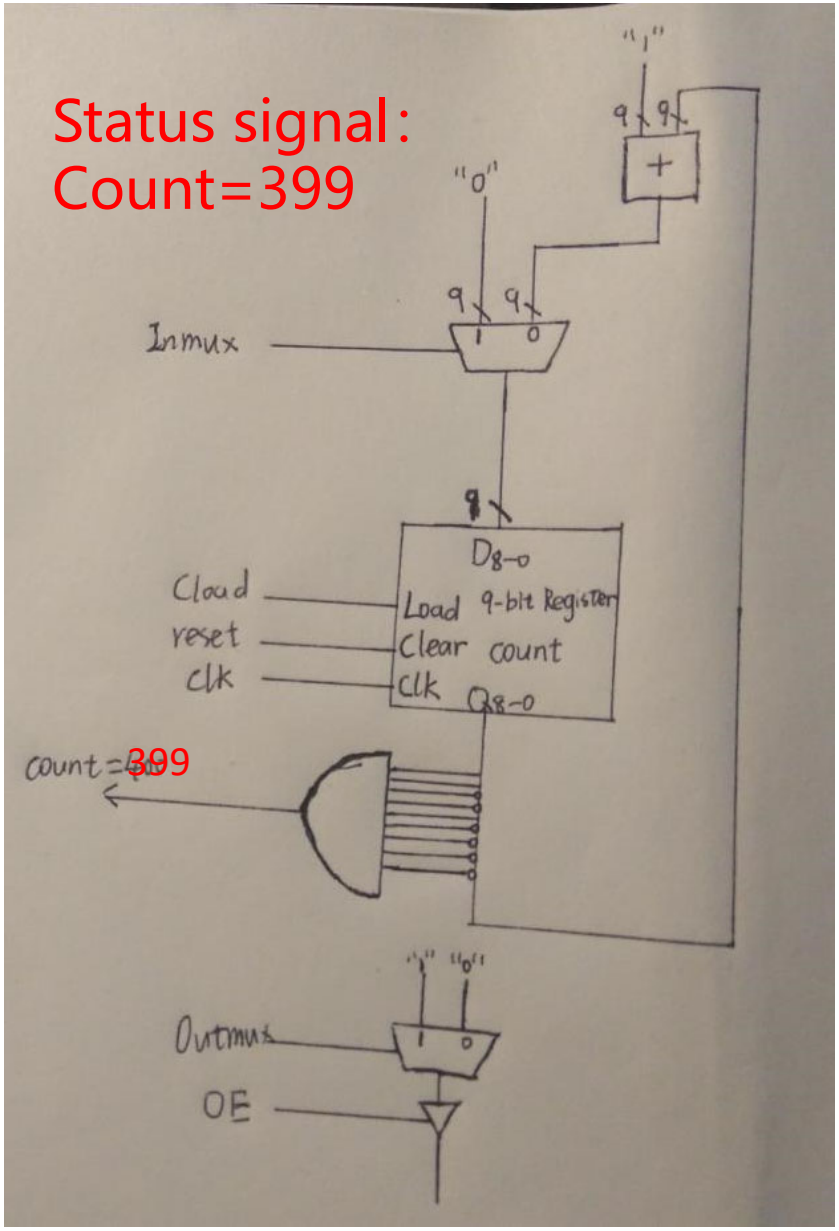
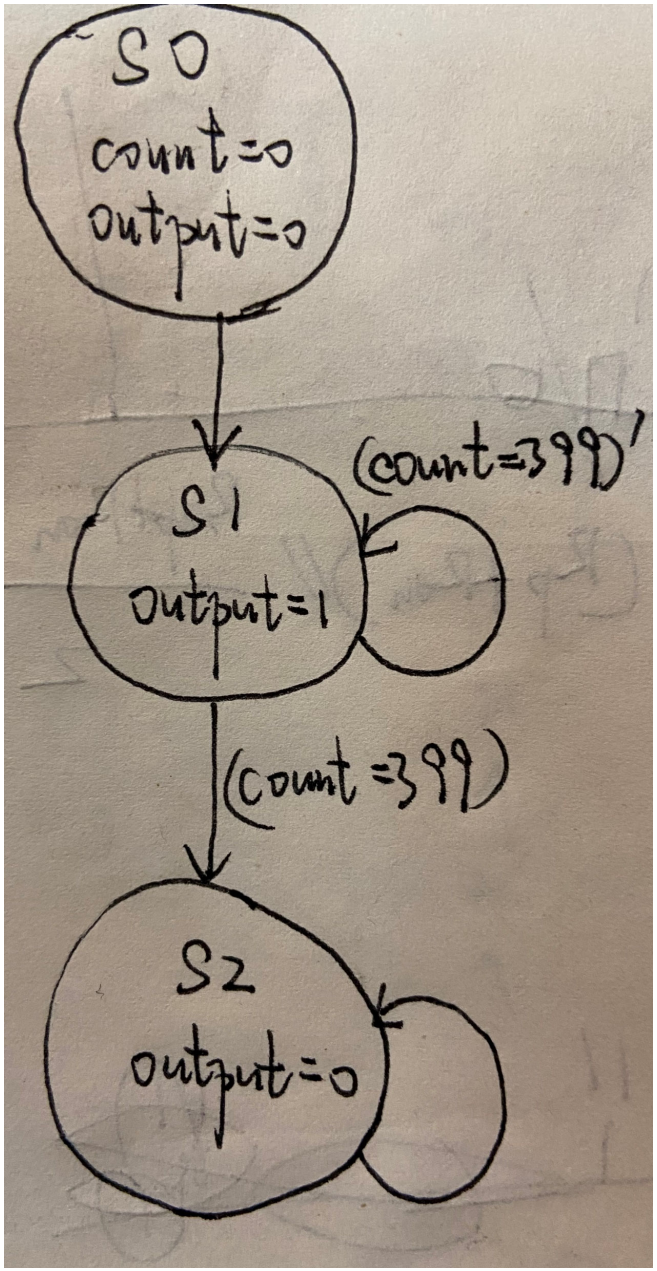
homework5、lab6

1. Construct a FSM circuit that will turn on a LED for 100 seconds and then turn it off. Assume that the input clock frequency is 4 Hz. This means that for every second, the FSM will go through four states. You cannot use a clock divider to slow down the clock. If you just use the number of states to do the delay, this will require 400 states. This will require too many flip-flops for drawing the circuit manually. So instead, you can use a counter to count how many times to loop around in a state before going on to the next state to get this time delay.



Control Word	Instruction	Inmux	Cload	Outmax	OE
1	Count=0	1	1	0	0
2	Count=count+1 OUTPUT 1	0	1	1	1
3	OUTPUT 0	0	0	0	1

$(399)_{10=}(1\ 1000\ 1111)_2$



Current State Q1Q0	Implementation D1D0	
	(count=399)’	(count=399)
S0 00	01	01
S1 01	01	11
S2 11	11	11
Unused 10	00	00

D1 = Q1Q0+Q0(count=399)
D0 = Q1'+Q0

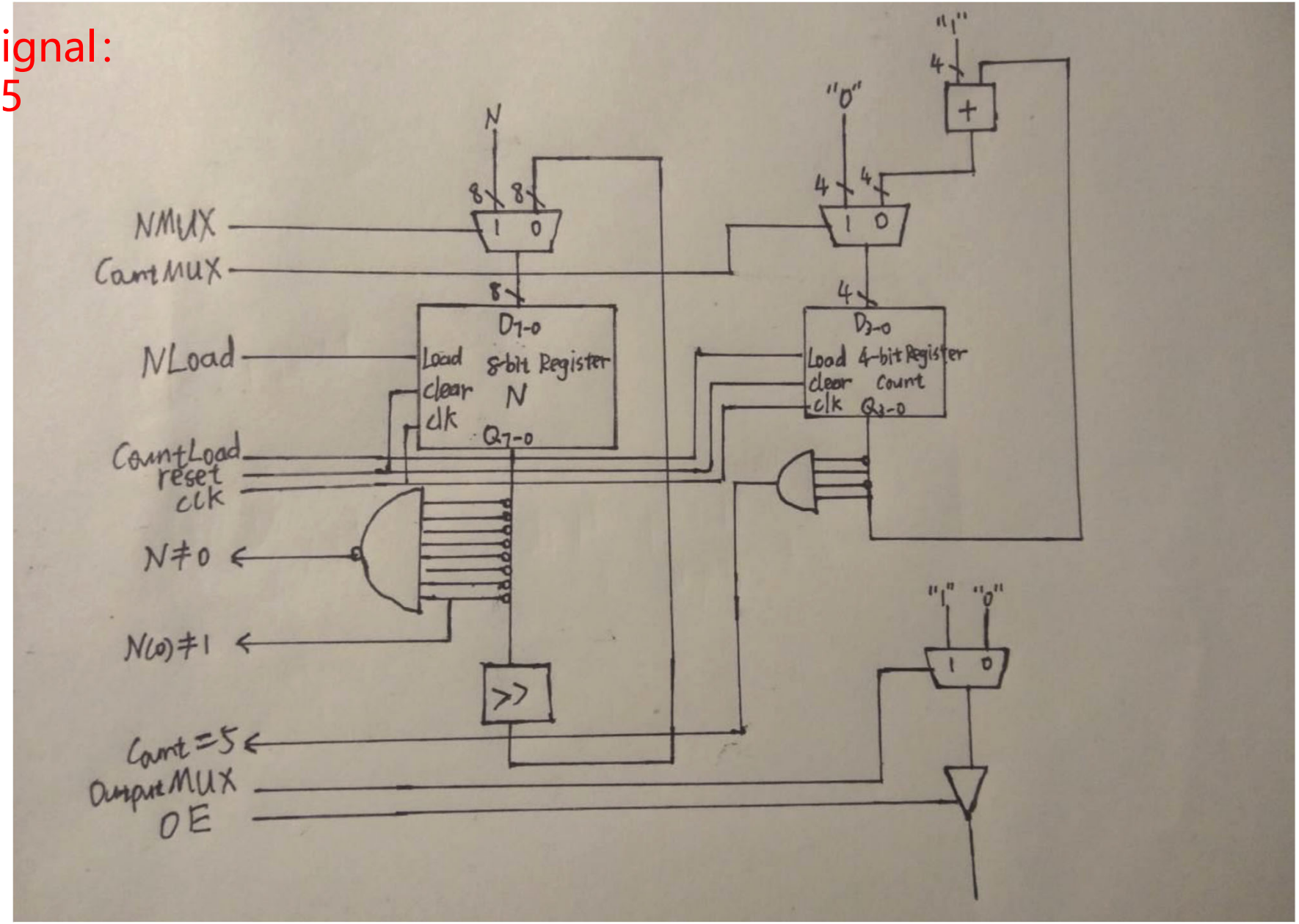
Q1Q0	Instruction	Inmux	Cload	Outmax	OE
00	Count=0	1	1	0	0
01	Count=count+1 OUTPUT 1	0	1	1	1
11	OUTPUT 0	0	0	0	1
10	No operation	0	0	0	0

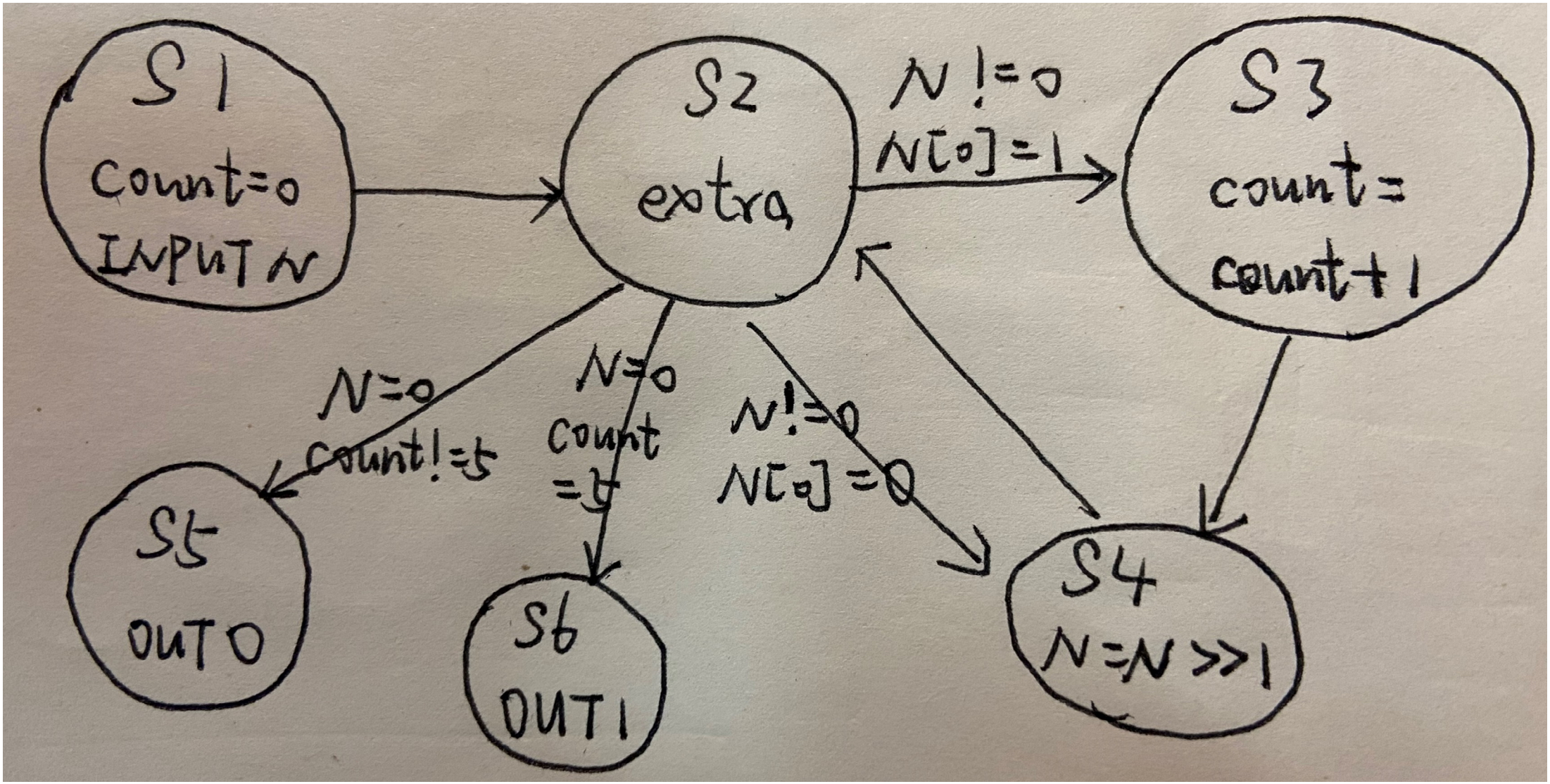
Inmux=Q1'Q0' Outmax= Q1'Q0
Cload=Q1' OE=Q0

2. Manually design and implement on a FPGA a dedicated microprocessor to enter one 8-bit number. Output a 1 if the number has five 1 bits; otherwise, output a 0.

```
1   Count = 0   // for counting the number of 1 bits
2   INPUT N
3   WHILE (N ≠ 0){
4     IF (N(0) = 1) THEN // least significant bit of N
5       Count = Count + 1
6     END IF
7     N = N >> 1          // shift N right one bit
8   }
9   OUTPUT (Count = 5)   // output 1 if the test (Count = 5) is true
```

Status signal:
Count=5
N≠0
N[0]≠1



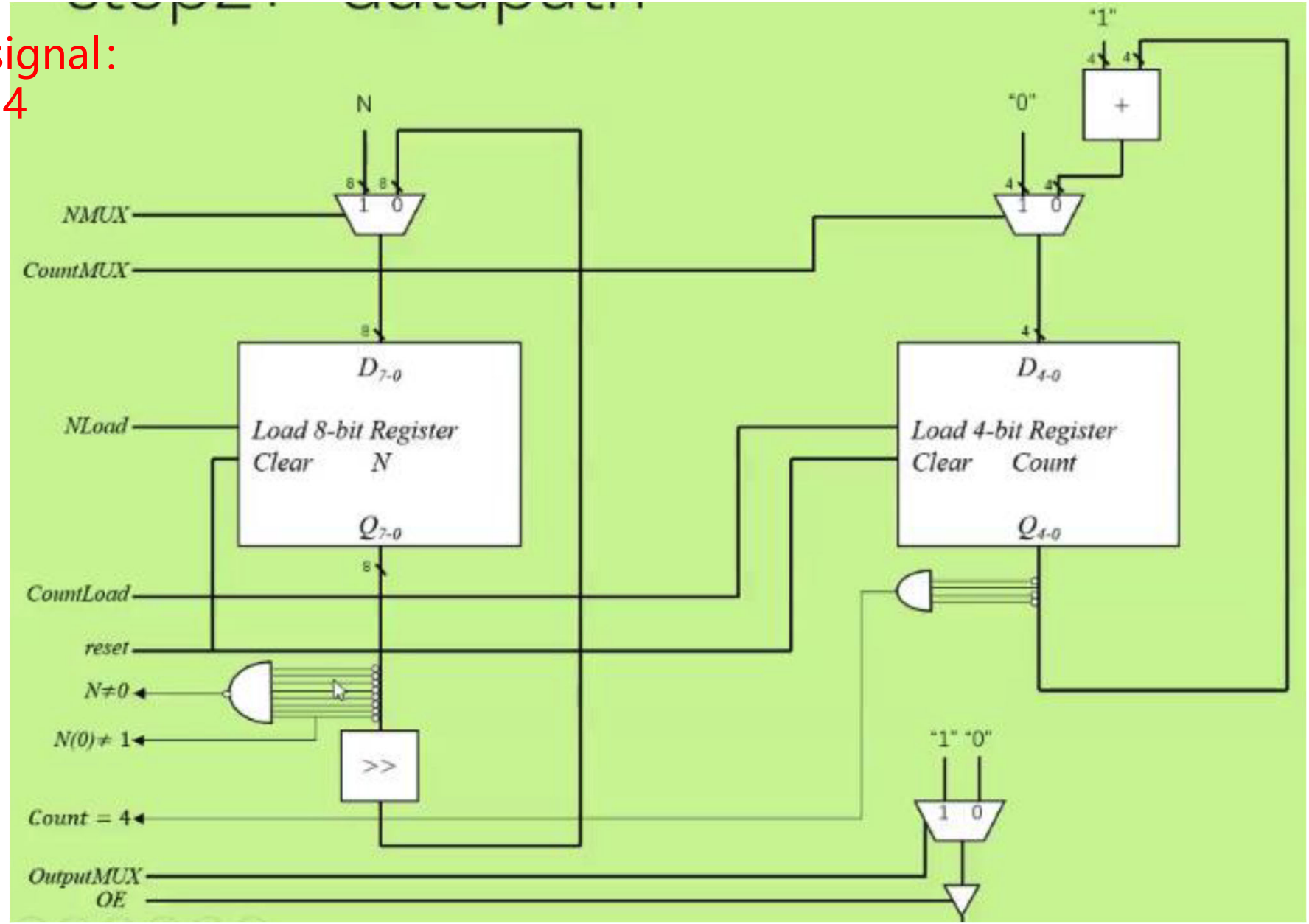

S1
Count=0
INPUT N
S2
extra
N!=0
N[0]=1
S3
count=
count+1
N=0
Count!=5
N=0
Count
=5
N!=0
N[0]=0
S5
OUT 0
S6
OUT 1
S4
N=N>>1

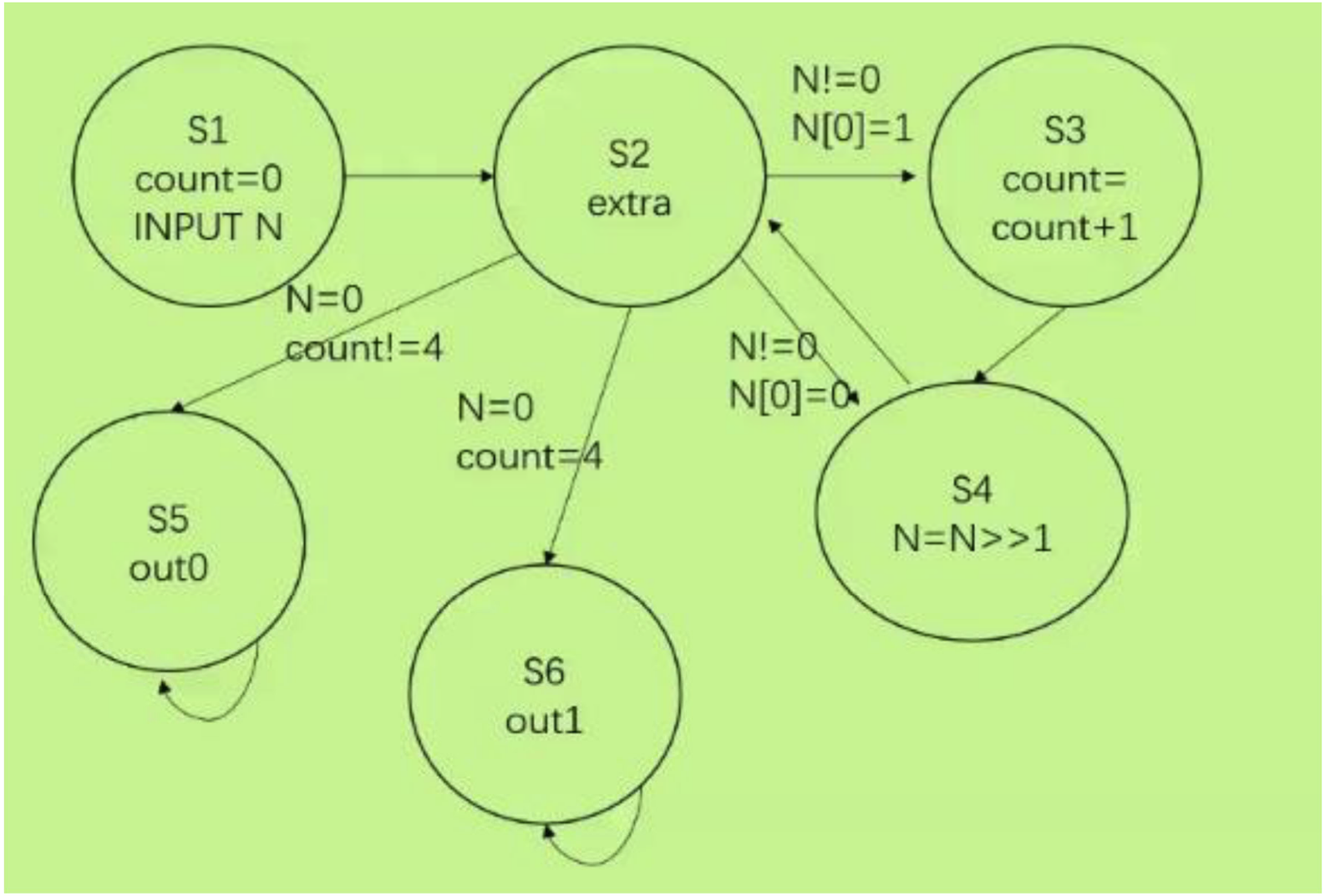
Control Word	Instruction	Nmux	Nload	Countmax	Countmax	OutputMux	OE
S1 001	Count=0 INPUT N	1	1	1	1	0	0
S2 010	No operation	0	0	0	0	0	0
S3 011	Count=Count+1	0	0	0	1	0	0
S4 100	N=N>>1	0	1	0	0	0	0
S5 101	OUTPUT 0	0	0	0	0	0	1
S6 110	OUTPUT 1	0	0	0	0	1	1
Unused 000	No operation	1	1	1	1	0	0
Unused 111	No operation	1	1	1	1	0	0

3. Also lab6. Manually design and implement on a FPGA a dedicated microprocessor to input one 8-bit value, and then determine whether the input value has an equal number of 0 and 1 bits. The microprocessor outputs a 1 if the input value has the same number of 0’ s and 1’ s; otherwise, it outputs a 0. (Similar to question2)

```
1   Count = 0   // for counting the number of 1 bits
2   INPUT N
3   WHILE (N ≠ 0){
4     IF (N(0) = 1) THEN // least significant bit of N
5       Count = Count + 1
6     END IF
7     N = N >> 1          // shift N right one bit
8   }
9   OUTPUT (Count = 4)   // output 1 if the test (Count = 4) is true
```

Status signal:
Count=4
N≠0
N[0]≠1




S1
count=0
INPUT N
S2
extra
N!=0
N[0]=1
S3
count=
count+1
N=0
count!=4
N=0
count=4
N!=0
N[0]=0
S5
out0
S6
out1
S4
N=N>>1

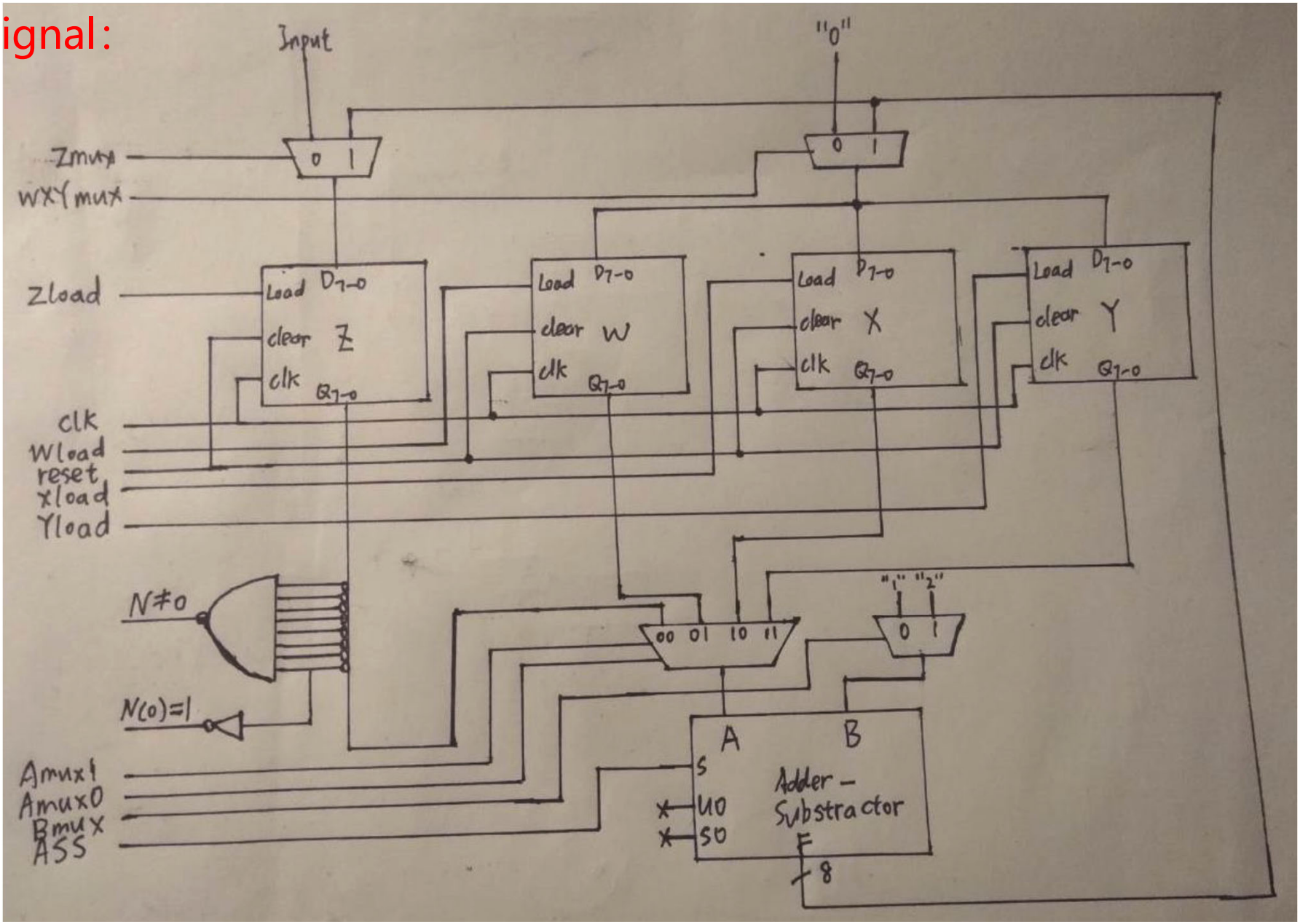
Control Word	Instruction	Nmux	Nload	Countmax	Countmax	OutputMux	OE
S1 001	Count=0 INPUT N	1	1	1	1	0	0
S2 010	No operation	0	0	0	0	0	0
S3 011	Count=Count+1	0	0	0	1	0	0
S4 100	N=N>>1	0	1	0	0	0	0
S5 101	OUTPUT 0	0	0	0	0	0	1
S6 110	OUTPUT 1	0	0	0	0	1	1
Unused 000	No operation	1	1	1	1	0	0
Unused 111	No operation	1	1	1	1	0	0

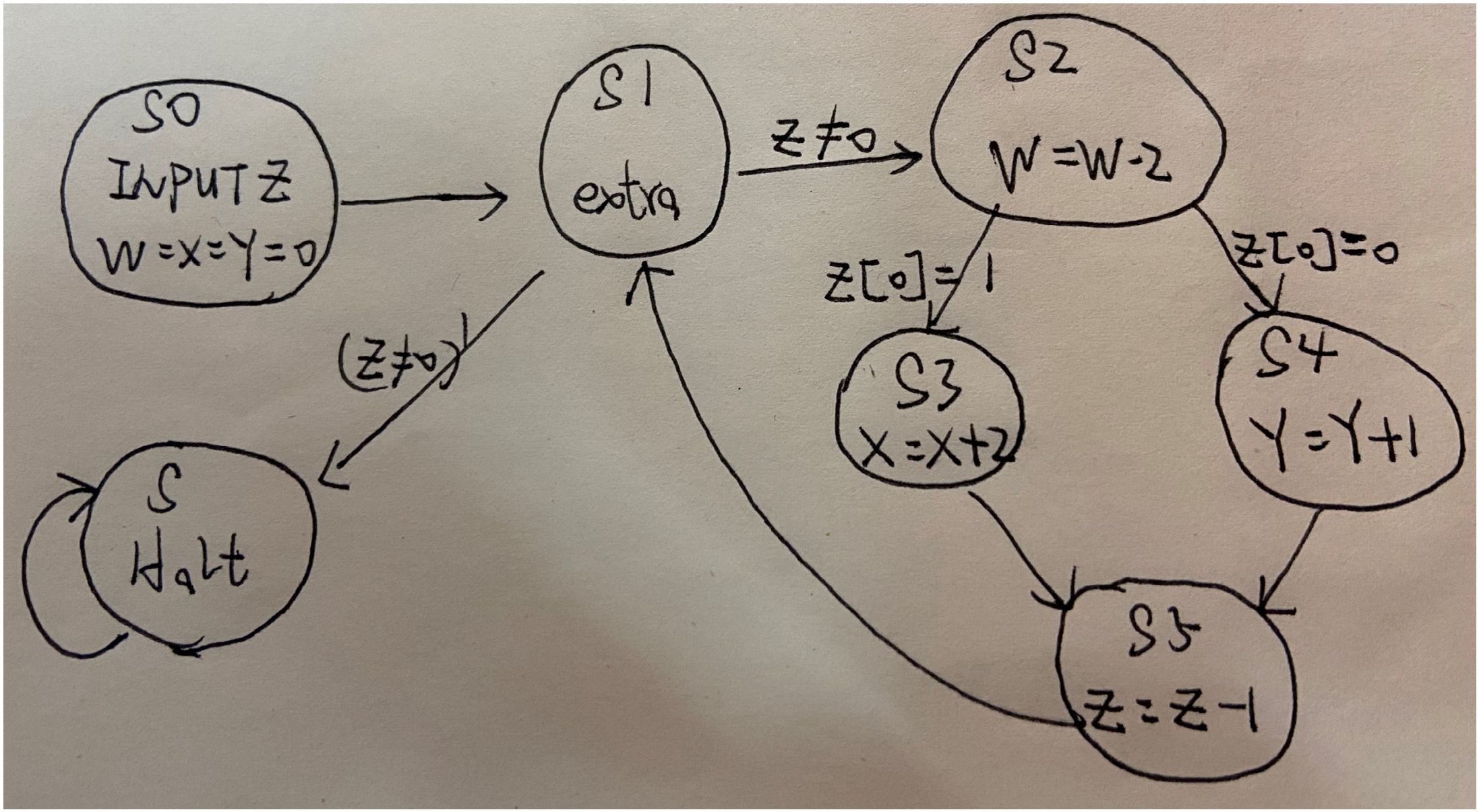
It is the same as question 2

4. Manually design and implement on a FPGA a dedicated microprocessor for executing the algorithm shown next. Use only one adder-subtractor unit for all of the addition and subtraction operations. Label clearly all of the control and status signals.

```
w = 0
x = 0
y = 0
INPUT z
WHILE (z ≠ 0) {
   w = w − 2
   IF (z is an odd number) THEN
     x = x + 2
   ELSE
     y = y + 1
   END IF
   z = z − 1
   }
```

Status signal:
Z≠0
Z[0]=1




S0
INPUT Z
W=X=Y=0
S1
extra
Z≠0
S2
W=W-2
Z[0]=1
Z[0]=0
(Z≠0)
S3
X=X+2
S4
Y=Y+1
S
Halt
S5
Z=Z-1

Control Word	Instruction	Zmux	WXYmux	Zload	Wload	Xload	Yload	Amux	Bmux	ASS
S0 000	INPUT Z W=X=Y=0	0	0	1	1	1	1	00	0	0
S1 001	No operation	0	0	0	0	0	0	00	0	0
S2 010	W=W-2	0	1	0	1	0	0	01	1	1
S3 011	X=X+2	0	1	0	0	1	0	10	1	0
S4 100	Y=Y+1	0	1	0	0	0	1	11	0	0
S5 101	Z=Z-1	1	0	1	0	0	0	00	0	1
S6 110	No operation	1	1	1	1	0	0	00	0	0
Unused 111	No operation	0	1	0	0	0	0	00	0	0